gather/binomial

An AgentMPI collective scaling analysis

Abstract

The binomial gather folds the fan-in into a tree: a rank with its lowest bit set sends its accumulated subtree to the parent obtained by clearing that bit, so contributions merge on the way up and the root performs $\lceil \log_2 p \rceil$ receives instead of $p - 1$. It moves exactly the same $p - 1$ messages as the linear gather — confirmed at all twenty-one measured sizes, 7 at $p = 8$ and 31 at $p = 32$ — and the round count matches $\lceil \log_2 p \rceil$ at every size, stepping only at 2, 4, 8, 16 and 32. What it trades is volume: the ranks nearest the root forward their whole subtree, so total traffic rises from $(p - 1)n$ to $np\lceil \log_2 p \rceil/2$, 80 units against 31 at $p = 32$. The family view turns up one real inaccuracy in that volume term. It is exact only when p is a power of two; at every other size it over-predicts, by 50 % at $p = 3$ and $p = 5$, 33 % at $p = 10$, 25 % at $p = 20$, because the subtrees are truncated by p. The implementation's self-report equals the exact truncated count at all thirteen distinct sizes, so the formula is the approximation, not the code. The payoff of the tree is measured at $p = 32$ as a fall in root blocking from 0.255 s to 0.049 s, a factor of 5.2 — and in the real eight-rank software run, where this is the algorithm that actually ran, the same structure carried 29,847 tokens to the root in seven rendezvous messages while the root's context high-water stayed at 0.

1 What this family is

The `gather` collective under the `binomial` algorithm, measured at 21 process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- **[ok]** All 21 measured sizes logged exactly the number of messages the closed form predicts.
- **[note]** Wall times span 0.005–0.107 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

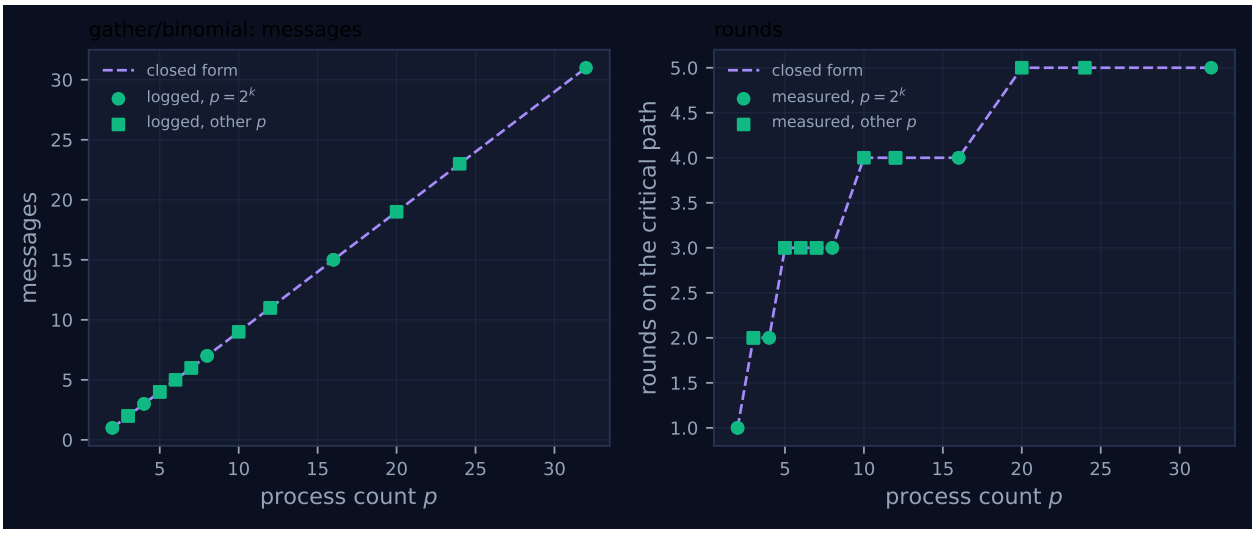

Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective's own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	`mb-free`	1	1	1	1	1	0.012	✓	✓
2	`mb-smoke`	1	1	1	1	1	0.013	✓	✓
3	`mb-free`	2	2	2	2	2	0.012	✓	✓
3	`mb-smoke`	2	2	2	2	2	0.005	✓	✓
4	`mb-free`	3	3	3	2	2	0.022	✓	✓
4	`mb-smoke`	3	3	3	2	2	0.023	✓	✓
5	`mb-free`	4	4	4	3	3	0.023	✓	✓
5	`mb-smoke`	4	4	4	3	3	0.023	✓	✓
6	`mb-free`	5	5	5	3	3	0.024	✓	✓
7	`mb-free`	6	6	6	3	3	0.058	✓	✓
7	`mb-smoke`	6	6	6	3	3	0.025	✓	✓
8	`mb-free`	7	7	7	3	3	0.031	✓	✓
8	`mb-smoke`	7	7	7	3	3	0.054	✓	✓
10	`mb-free`	9	9	9	4	4	0.071	✓	✓
12	`mb-free`	11	11	11	4	4	0.036	✓	✓
12	`mb-smoke`	11	11	11	4	4	0.044	✓	✓
16	`mb-free`	15	15	15	4	4	0.045	✓	✓
16	`mb-smoke`	15	15	15	4	4	0.055	✓	✓
20	`mb-free`	19	19	19	5	5	0.065	✓	✓
24	`mb-free`	23	23	23	5	5	0.077	✓	✓
32	`mb-free`	31	31	31	5	5	0.107	✓	✓

4 How the algorithm scales

The tree is the binomial tree, expressed in bit arithmetic on the rank's position relative to the root. Each rank computes its virtual rank $vr = (r - \text{root}) \bmod p$ and holds a dictionary `acc` keyed by virtual rank, initially just its own contribution. It then walks mask $= 1, 2, 4, \ldots$: if vr has that bit

set, it sends the whole of `acc` to the parent $vr - \text{mask}$ and stops; otherwise, if the child $vr|\text{mask}$ exists, it receives that child's accumulated dictionary and merges it. The root ($vr = 0$) never sends, so its walk runs through every mask below p, receiving from $\lceil \log_2 p \rceil$ children.

Three costs follow. Every non-root rank sends exactly once, so the message count is $p-1$ — identical to the linear gather, which is the point of the comparison. The critical path is the height of the tree, $\lceil \log_2 p \rceil$, because a rank cannot forward until its children have arrived. And the volume is not $(p-1)n$: a rank that sends at mask m sends its entire subtree, which has m members when the tree is complete, so summing over ranks gives $np\lceil \log_2 p \rceil/2$ — each of the $\lceil \log_2 p \rceil$ levels contributing about $np/2$. That is `FORMULAS["gather","binomial"]`, and the docstring for the mirror-image scatter states the trade explicitly: "the total volume forwarded is `n*p*log(p)/2` rather than the `n*(p-1)` of the linear algorithm — the price of the logarithmic depth".

Both checked terms agree everywhere. The message panel of Figure 1 is the line $p - 1$ at all twenty-one rows, and the round panel is the staircase $\lceil \log_2 p \rceil$, stepping at $p = 2, 4, 8, 16, 32$ and holding flat between: 3 across $p = 5, 6, 7, 8$, 4 across $p = 10, 12, 16$, 5 across $p = 20, 24, 32$. The steps land where they should because the loop bound is `while mask < p`, so the number of levels is $\lceil \log_2 p \rceil$ by construction rather than by coincidence.

What the two panels cannot show is where the waiting went, and that is the measurement that justifies the algorithm. At $p = 32$ the root performs 5 receives totalling 0.049 s of blocking, against 31 receives totalling 0.255 s under `linear` — a 5.2-fold reduction in the root's exposure. The collective's aggregate blocking is higher, not lower: 0.46 rank-seconds against 0.26, because interior ranks now wait for their children too. That is exactly the trade the tree makes, and stating it as an aggregate reduction would be wrong. What the tree does is *redistribute* the waiting away from the single rank that cannot be replicated.

The message log gives the shape exactly. Every one of the 31 non-root ranks sends once, and the 31 receives are distributed as 5 at rank 0, 4 at rank 16, 3 each at ranks 8 and 24, 2 each at ranks 4, 12, 20 and 28, and 1 each at the eight even ranks $2, 6, 10, \ldots, 30$ — a rank's fan-in being the number of trailing zeros in its virtual rank. The viewer screenshot for `mb-free__coll-gather-binomial-32` shows the same distribution as lane structure: rank 0's lane carries five or six glyphs rather than a dense run of bars, the interior ranks two to five each, and the sixteen odd ranks one — their single send. Beside `mb-free__coll-gather-linear-32`, where one lane has everything and thirty-one have one tick each, the difference between $O(p)$ and $O(\log p)$ fan-in at the root is visible without reading a number.

5 Powers of two and everything else

This is a bit-arithmetic algorithm, so the non-power-of-two path is where a defect would live, and it needs two separate questions: does the implementation behave, and does the closed form describe it?

The implementation behaves. At a non-power-of-two size the tree is the power-of-two tree with the out-of-range nodes pruned: the guard `if child_v < p` suppresses receives from children that do not exist, and the parent computation $vr - (vr \wedge -vr)$ is valid for any vr. The consequence is that some subtrees are smaller than their mask suggests — at $p = 5$, rank 4 is a child of rank 0 at mask 4 but has no subtree of its own — and the message count is unaffected, because every non-root rank still sends exactly once. The measurements bear that out at all twenty-one sizes: 2, 4, 5, 6, 9, 11, 19 and 23 messages at $p = 3, 5, 6, 7, 10, 12, 20, 24$, with round counts 2, 3, 3, 3, 4, 4, 5 and 5, matching

$\lceil \log_2 p \rceil$ throughout. No red crosses appear, so the self-reported count equalled the logged traffic everywhere, and the eight sizes measured in both campaigns agree on every integer.

The closed form does not describe it exactly, and this is the defect the family view exposes. The volume term assumes complete subtrees. The exact traffic is $\sum_{vr=1}^{p-1} |\{x : vr \le x < \min(vr + (vr \wedge -vr),\, p)\}|$ — the sum of *truncated* subtree sizes — and that equals $p\lceil \log_2 p \rceil / 2$ only when p is a power of two. Comparing the exact count, the closed form, and the volume the collective reported in each run:

p	reported (units)	exact	closed form	ratio
2	1	1	1.0	1.000
3	2	2	3.0	0.667
4	4	4	4.0	1.000
5	5	5	7.5	0.667
6	7	7	9.0	0.778
7	9	9	10.5	0.857
8	12	12	12.0	1.000
10	15	15	20.0	0.750
12	20	20	24.0	0.833
16	32	32	32.0	1.000
20	40	40	50.0	0.800
24	52	52	60.0	0.867
32	80	80	80.0	1.000

The reported volume equals the exact truncated count at every one of the thirteen sizes, and equals the closed form at exactly the five powers of two. Everywhere else the closed form over-predicts, worst at $p = 3$ and $p = 5$ where it is 1.5 times the truth. The signature — exact at $2, 4, 8, 16, 32$ and a systematic excess at every other size — is the same one the repository already documents for recursive-doubling allreduce, and it is invisible in any single run for the same reason: one run has one p and nothing to compare against.

Two things distinguish it from that earlier defect and both should be said plainly. It is in the *volume* term, which the generated table above does not check: the table validates `logged` against `pred.` for messages and `rounds` against `pred.` for rounds, and volume appears in neither column, which is why the family carries a **[ok]** rather than a warning. And it errs conservatively — the model over-states cost, so a harness relying on it would over-provision — which is the opposite of the allreduce case, where the count made the collective look cheaper than it was. It still merits a fix, because `predict` derives `price_usd` from volume and a 50 % over-estimate of tokens at $p = 5$ is a 50 % over-estimate of the bill. The exact expression is a one-line sum and `scatter/binomial` needs the identical correction, its volumes being the same numbers by duality.

6 When to choose this algorithm

Against `linear` the comparison is unusually clean, because the message counts are identical at every size in both families — $p - 1$, twenty-one rows each — so nothing is traded in traffic and the whole difference is depth against volume. Depth: $\lceil \log_2 p \rceil$ against $p - 1$, or 5 against 31 at $p = 32$. Volume: 80 units against 31 at $p = 32$, a factor of 2.6. `cost.predict` at $n = 4{,}096$ tokens per contribution prices the two at 2.58 s against 15.97 s of critical path and 327,680 against 126,976 tokens, and `cost.best_algorithm` answers `binomial` on the time objective and `linear` on the volume objective at every (p, n) I evaluated. The two objectives genuinely disagree, and which one binds is a property of the harness rather than of the collective.

For agent ranks the tie-breaker is that the root is not replicable. Under `linear` the root performs $p-1$ receives; if the root is an agent, that is $p-1$ turns during which the harness makes no progress, and it is a serial fraction in Amdahl's sense that does not shrink with p. Under `binomial` it is $\lceil \log_2 p \rceil$ turns, and the measured root blocking at $p = 32$ falls from 0.255 s to 0.049 s. The volume penalty, meanwhile, is only a real cost if the bodies move by value — and this is where the transport story matters. In `real-sw-p8-full` the `collect-r1` gather ran exactly this algorithm at $p = 8$: all seven sends were `mode: rendezvous`, so the 29,847 tokens it accounted for travelled as handles; the interior rank 4 forwarded a 13,206-token block that was its own contribution plus its two children's (5,269 and 4,436), and every receive is logged `admitted: false` with `admitted_tokens: 0`. Every rank finalised with a context high-water of 0 against a 120,000-token budget. So the binomial gather's volume penalty was paid in references, and the $np \log_2 p/2$ term never touched an agent's context. That is the case for the tree: take the depth reduction, and pay the volume in handles rather than in tokens.

The property that makes the trade safe is worth naming, since it is the one an implementation could easily lose. Interior ranks forward other ranks' artefacts, and the branch receives them with `admit=False` hard-coded, so no interior agent reads work that belongs to someone else. That is a confidentiality guarantee as much as a context-budget one. It has a sharp edge, however: because the literal `False` replaces the caller's argument, the `admit` keyword is silently ignored on this path while `linear` honours it. Since `gather()` defaults to `"linear" if p <= 4 else "binomial"`, `comm.gather(x, admit=True)` charges the root's context at $p = 4$ and does not at $p = 5$. The sweep cannot show this — it names the algorithm and leaves `admit` at its default, so all twenty-one runs log `admitted: false` — but a harness that relies on `admit=True` to place contributions in the root's context will find it works at small p and stops working when the default flips.

7 Threats to this reading

No agents took part. No executor is recorded, the viewer reports zero agent calls, and the 0.005–0.107 s wall times are SQLite writes and event-log appends. The blocking figures — 0.049 s at the root, 0.46 rank-seconds aggregate — are fabric behaviour under thread contention. They establish the *shape* of the fan-in, which is the algorithm's contribution, and nothing about what a fan-in costs when each contribution is a model's output.

The contributions are one token each, which is what makes the volume comparison tractable and also what makes it small. At $p = 32$ the whole collective moves 80 tokens, so the 2.6-fold volume penalty is 49 tokens. Every statement above about context pressure comes from the closed form and from the real run; this family measures the pattern, not the pressure. The fabric's own `msg.send` records report a much larger figure than the collective's self-report — 544 tokens against 80 at $p = 32$ — because the fabric tokenises the whole `{"3": ..., "7": ...}` dictionary envelope while the collective counts only the contributions. With one-token payloads the envelope dominates entirely, so that 6.8-fold ratio is an artefact of the benchmark and would shrink towards nothing with realistic artefacts. I have used the payload figure throughout for comparability with the closed form.

The volume discrepancy is arithmetic I performed, not something the tooling flagged. The exact column above is a truncated-subtree sum evaluated over the measured grid; the reported column is `tokens_sent` summed over each run's `coll.gather` records. If the self-report were itself wrong in the same direction the agreement would be two errors cancelling. Against that: the two

are computed by different code from different data — the self-report sums `_tok()` over the values actually placed in each message — and they agree to the unit at all thirteen sizes, including the awkward ones where the truncation bites hardest.

The round count is asserted rather than observed. `tr.stats.rounds = _ilog2_ceil(p)` is computed from the same expression the closed form uses, so the agreement in that column is a consistency check between two statements of one intent. The independent evidence for the depth is the per-rank message pattern — the root receiving 5 messages at $p = 32$, the odd ranks sending one and receiving none — which is consistent with a tree of height 5 and inconsistent with a flat fan-in.

The `admit` asymmetry is read from source, not measured. No run in this sweep passes `admit=True`, so the claim that this branch ignores it rests on the literal `False` in the call and the pass-through in the `linear` branch. A test at $p = 4$ and $p = 5$ would settle it.